Secretary-Style Split Search for Decision Trees

Mathias Valla[a]

[a] *Chaire DIALog, Institut Louis Bachelier, Paris, France*

Abstract

The standard CART splitter examines all admissible covariate–threshold pairs at each node, which can make tree induction computationally expensive when many continuous features are available. This paper asks whether such exhaustive split enumeration is always necessary to grow accurate decision trees. We study a small family of secretary-style split-search rules that evaluate only a random subset of candidate splits and stop early according to simple sequential criteria. The proposed strategies include a split-level secretary rule with randomized threshold exploration, a double secretary rule over covariates and thresholds, a covariate-level secretary rule with exhaustive within-covariate refinement, and a parametric secretary rule calibrated from exact gain evaluations. We also evaluate a blockwise rank-inspired heuristic motivated by the block-rank algorithm.

The methods are compared with the standard exhaustive `scikit-learn` splitter and with ExtraTrees-style random-threshold baselines using several `max_features` budgets, on a broad collection of regression and classification datasets from the PMLB benchmark suite. After aligning secretary exploration with ExtraTrees-style continuous threshold draws and optimizing the exact-selection phase, the picture becomes more nuanced. Split-level secretary and prophet variants can now preserve predictive performance much more closely than before, but they do so with little or no wall-clock gain. The methods that achieve the largest practical speedups remain ExtraTrees, covariate-level secretary, and block-rank. Secretary-style split search is therefore best viewed as a family of trade-offs between exactness and runtime, rather than as a universal replacement for exhaustive or ExtraTrees-style split search.

Keywords: decision trees, CART, secretary problem, randomized split search, benchmark evaluation, computational efficiency

1. Introduction

Classification and regression trees remain widely used for tabular data because they combine flexibility, interpretability, and strong empirical performance [1]. Their standard induction rule is greedy: at each internal node, CART evaluates candidate covariate–threshold pairs and selects the split maximizing the impurity decrease. For continuous covariates, the number of admissible thresholds can be large, so exhaustive split search may dominate training time.

This raises a basic question: is it always necessary to evaluate all admissible splits in order to grow an accurate tree? In many nodes, only a small subset of candidates attains gains close to the optimum. Existing randomized tree procedures exploit this idea indirectly. Random forests subsample covariates [2], and extremely randomized trees randomize threshold choice [3]. These methods, however, are not formulated as stopping rules on a stream of candidate gains.

We recast split search as a secretary-style online selection problem. In the classical secretary problem, items arrive sequentially in random order and the decision maker must stop online to select a highly ranked item [4, 5]. The classical skip rule, with asymptotic success probability $1/e$, shows that a short exploration phase can be followed by a simple acceptance rule with a nontrivial guarantee [4, 5]. More recent work has introduced rank-adaptive rules based on the availability of a single sample, including the block-rank family [6]. These ideas suggest a natural analogue for tree induction: if covariates and thresholds are explored in random order, split selection becomes a stopping problem on a stream of exact impurity gains.

We study four secretary-style split-search strategies for CART, ranging from split-level stopping to covariate-level stopping and model-based calibration, and also include a blockwise rank-inspired variant and a one-sample prophet-style baseline.

The contribution is empirical and methodological rather than theoretical. We benchmark the proposed rules against the standard exhaustive `scikit-learn` splitter and against ExtraTrees-style random-threshold baselines [3] on a large collection of regression and classification datasets from the PMLB benchmark suite [7]. The goal is to characterize the time–accuracy trade-off, quantify the variability induced by randomized split exploration, and identify which stopping rules offer the best practical compromise at the single-tree level before studying ensembles built from the same splitters. An open-source implementation of all methods and the scripts used to reproduce the experiments are released publicly.[1]

2. Methods

2.1. Split-search setting

Consider a node $\mathcal{N}$ containing n observations $\{(X_i, Y_i)\}_{i=1}^n$ with $X_i \in \mathbb{R}^p$. Let $\mathcal{J} = \{1, \ldots, p\}$ be the set of covariates, and let $\mathcal{S}_j$ be the set of admissible thresholds for covariate j. A candidate split is a pair (j, s) with $s \in \mathcal{S}_j$, producing children of sizes $(n_{\mathrm{L}}, n_{\mathrm{R}})$. For an impurity I, the gain of split (j, s) is

$$G_{j,s} = I(\mathcal{N}) - \frac{n_{\mathrm{L}}}{n} I(\mathcal{N}_L) - \frac{n_{\mathrm{R}}}{n} I(\mathcal{N}_R). \tag{1}$$

For regression, I is the mean squared error impurity. For classification, we consider Gini and entropy impurities [1]. Once a candidate threshold is chosen, its gain is computed exactly on the full node data. In the optimized implementations studied here, randomness comes from the order in which covariates are visited and, for split-level exploration, from ExtraTrees-style continuous threshold draws within visited covariates.

To study early stopping, we impose a random order on visited covariates and use simple node-size-dependent exploration budgets. In the experiments, the exploration budget at a node is parameterized as a function of node size n, with schedules of order n/e, $0.1n$, $\log n$, and $\sqrt{n}$. The acceptance stage then combines these randomized exploration draws with exact within-covariate scans.

2.2. Stopping rules

Method S uses a random covariate order together with a split-level exploration budget. A subset of visited covariates is used for exploration; on each such covariate, a small number of ExtraTrees-style continuous thresholds is sampled and the best resulting gain updates the exploration maximum. Subsequent covariates are then scanned exactly, and the first covariate whose best exact split beats the exploration maximum is accepted. If no such covariate appears, the best exploration split is returned. This is the optimized tree-split analogue of the classical skip–then–accept secretary rule [4, 5].

[1] Code available at https://github.com/MathiasValla/EarlyStoppingTrees

Method S_{all} applies the secretary rule at the covariate level. For a visited covariate j, its score is

$$V_j = \max_{s \in \mathcal{S}_j} G_{j,s}.$$

Covariates are processed in random order, and a secretary rule is applied to the sequence (V_j). When a covariate is accepted, the best threshold within that covariate is returned. This method is more expensive than S for each inspected covariate, but it preserves exact within-covariate optimization.

Method S^2 introduces a nested secretary structure. The outer rule is applied to covariates in random order. For each visited covariate j, an inner exploration phase samples a small number of ExtraTrees-style continuous thresholds, and an exact within-covariate scan then returns the best threshold whose gain exceeds the inner exploration level. This yields a realized score $\tilde{V}_j$ for the covariate, and the outer rule is then applied to the sequence $(\tilde{V}_j)$. The outer stage selects the covariate and the inner stage selects the threshold.

Method S_{par} is a model-based variant. For a given covariate j, a small number of continuous random thresholds is first evaluated exactly, yielding gain samples used to fit a working distributional model or an empirical upper quantile. A covariate-specific acceptance threshold is then derived from this fit, and the covariate is represented by the first exact threshold whose gain exceeds the fitted level. The goal is not to claim an exact generative law for random-threshold gains, but to provide a calibrated stopping rule based on a small number of exact evaluations.

Inspired by the block-rank algorithm of Nuti and Vondrák [6], we also evaluate a blockwise rank-inspired heuristic. For a visited covariate, admissible thresholds are scanned in contiguous blocks of size about $\sqrt{n}$, each block is summarized by its exact best gain, and a secretary-style rule is applied to the resulting stream of block maxima. This is not intended as a literal transcription of the original block-rank schedule, but rather as a tree-specific blockwise analogue motivated by the same idea of using coarser rank information than full exhaustive search.

We also include a one-sample prophet-style baseline. For each visited covariate, one ExtraTrees-style continuous threshold is sampled and evaluated exactly, and the maximum of these gains is used as an empirical acceptance threshold. A second pass then scans the visited covariates exactly and returns the first covariate admitting a split above this threshold, with the original exploration split used as a fallback if no such covariate appears. This baseline is motivated by prophet inequalities from samples [8, 9].

2.3. Motivating heuristics

This subsection is motivational only. We do not attempt a formal analysis of tree induction under secretary-style exploration, and the paper's contribution remains empirical and methodological.

The classical secretary problem shows that, under a random arrival order, a simple skip-then-accept rule with exploration length of order N/e is asymptotically optimal for selecting the best item [4, 5]. This is the only theoretical fact used directly in the benchmark design: it motivates the default n/e exploration schedule that we treat as the representative secretary-style setting in the main comparison.

The parametric variant S_{par} was motivated more loosely. For a fixed split, regression gains under MSE admit an exact positive-valued law, whereas classification gains are often well approximated by chi-square- or gamma-like families under standard asymptotics. This suggests that a small sample of exact gain evaluations might be used to fit a working distribution and derive a conservative acceptance threshold [10]. In the present paper, however, this idea is used only as a heuristic calibration device; we do not claim that the fitted model is exact, nor do we derive guarantees for the resulting stopping rule.

All methods were implemented in a common open-source package built around a modified tree splitter, with identical tree-growing hyperparameters across methods. This ensures that the observed differences are attributable to split-search rules rather than to other changes in the induction procedure.

3. Results

3.1. Experimental protocol

We compared secretary-style split search against the standard exhaustive splitter of `scikit-learn` and against ExtraTrees-style random-threshold baselines with `max_features` set to 1, $p/3$, $2p/3$, and p, on a broad collection of regression and classification datasets from the PMLB benchmark suite [7]. For each dataset, the same 5-fold cross-validation splits and tree hyperparameters were used across methods. Because the proposed procedures are

Table 1: Summary of the PMLB benchmark used in the study.

Task	N	median n	median p	$n_{\min}$–$n_{\max}$	$p_{\min}$–$p_{\max}$
Regression	113	500	10	47–40768	2–124
Classification (Gini)	122	736	13	32–58000	2–240
Classification (Entropy)	122	736	13	32–58000	2–240

randomized, each dataset–method pair was evaluated over 50 independent runs with different estimator seeds. After filtering to datasets that completed successfully under the common protocol, the final benchmark retained 113 regression datasets and 122 classification datasets for each impurity criterion.

The primary computational metric is wall-clock training time. To make the timing results more interpretable, we also track split-search effort through the number of gain evaluations, either measured directly for the secretary-style splitters or reconstructed deterministically a posteriori for the exhaustive and ExtraTrees baselines. Predictive performance is measured by RMSE in regression and by F1-score in classification; effort plots, reliability summaries, and additional visualizations are reported in the supplementary material. Results are normalized relative to the exhaustive splitter through a speedup factor and a relative loss metric. Because different splitters can induce different tree shapes, total tree-level effort can occasionally exceed the exhaustive baseline on a minority of datasets even when the node-level rule evaluates fewer gains on any given feature.

3.2. Global trade-off

Figure 1 summarizes the main Pareto comparison between speedup and predictive degradation. Each point corresponds to one dataset summarized by the median over the 50 runs. Size-stratified versions of the same Pareto comparison are reported in Supplementary Figure S1. Three conclusions are immediate: the time–accuracy trade-off depends strongly on the task; the strongest speed-oriented baselines remain the ExtraTrees family; and the optimized split-level secretary rules now lie much closer to the exhaustive baseline in predictive loss than in runtime.

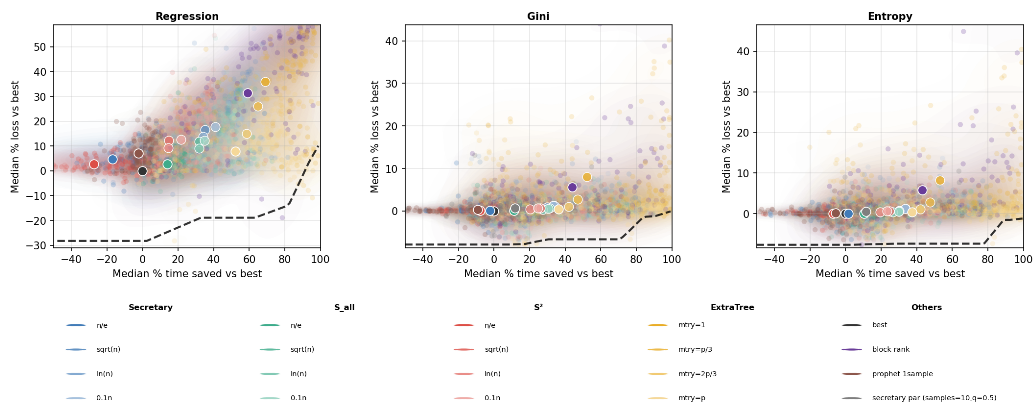

Figure 1: Global Pareto comparison of exhaustive, secretary-style, and ExtraTrees-style split-search methods. Each point represents one dataset summarized by the median over 50 runs. The horizontal axis reports the median percentage of training time saved relative to the exhaustive `scikit-learn` splitter, and the vertical axis reports the median percentage loss relative to the same baseline. Left: regression with RMSE loss. Middle: classification with Gini impurity and F1 loss. Right: classification with entropy impurity and F1 loss. Large white-edged markers indicate the cross-dataset medians of the corresponding methods.

Table 2 reports a representative benchmark summary. For the secretary families, we use the theoretically motivated n/e schedule; for the parametric strategy, we report the low-overhead classification configuration $(10, q = 0.5)$; and for ExtraTrees we report all four `max_features` budgets. In addition to speedup and loss, the table now reports gain-evaluation effort saved, which helps separate algorithmic budget reductions from residual implementation overhead.

The regression results remain unfavorable to aggressive approximation, but the optimized split-level rules change the interpretation. The strongest speed-oriented baselines are still ExtraTrees: ExtraTree($m_{\text{try}} = p$) achieves median speedup 2.62× with 8.7% median RMSE loss, while ExtraTree($m_{\text{try}} = 1$) reaches 5.56× speedup at a much larger 36.7% loss. By contrast, the optimized $S(n/e)$ rule now has only 0.5% median loss, and the one-sample prophet reaches zero median loss, but both operate essentially at exhaustive speed: $S(n/e)$ is slower than exhaustive on the median dataset and prophet is only marginally faster. This shows that the predictive gap of the split-level rules was not purely algorithmic, but also that exact replay and acceptance

Table 2: Representative global summary by method after the optimized 50-run rerun. The n/e schedule is used for the secretary families, the low-overhead $(10, q = 0.5)$ configuration is used for S_{par} in classification, and all four ExtraTrees `max_features` budgets are shown. Speedup is relative to the exhaustive `scikit-learn` splitter. Effort saved is the median percentage reduction in gain evaluations relative to exhaustive split search; negative values indicate methods whose replay logic performs more exact gain evaluations than the exhaustive baseline. Loss is expressed in percent.

Task	Method	Median speedup	Median effort saved (%)	Median loss (%)	90th pct. loss (%)	$P(\text{loss} \leq 5\%)$	$P(\text{speedup} \geq 20\%)$
Regression	Exhaustive	1.00	0.0	0.0	0.0	1.000	0.000
Regression	S (n/e)	0.53	-8.5	0.5	3.2	0.947	0.000
Regression	S^2 (n/e)	1.42	58.7	21.7	48.9	0.186	0.726
Regression	S_{all} (n/e)	2.30	70.6	20.5	48.6	0.204	0.956
Regression	Block-rank	2.80	90.8	33.0	55.0	0.159	0.938
Regression	1-sample prophet	1.03	59.6	0.0	9.9	0.717	0.053
Regression	ExtraTree ($m_{\text{try}} = 1$)	5.56	98.9	36.7	60.7	0.115	0.965
Regression	ExtraTree ($m_{\text{try}} = p/3$)	4.51	96.7	28.7	44.9	0.168	0.965
Regression	ExtraTree ($m_{\text{try}} = 2p/3$)	3.34	93.0	14.9	29.4	0.212	0.965
Regression	ExtraTree ($m_{\text{try}} = p$)	2.62	88.8	8.7	17.0	0.336	0.938
Classification (Gini)	Exhaustive	1.00	0.0	0.0	0.0	1.000	0.000
Classification (Gini)	S (n/e)	0.65	-13.5	0.0	0.9	0.992	0.000
Classification (Gini)	S^2 (n/e)	1.33	38.3	1.7	10.8	0.787	0.598
Classification (Gini)	S_{all} (n/e)	1.77	60.8	1.5	11.1	0.762	0.680
Classification (Gini)	S_{par} $(10, q = 0.5)$	0.90	26.7	0.3	3.4	0.951	0.107
Classification (Gini)	Block-rank	1.89	72.4	2.5	14.3	0.639	0.721
Classification (Gini)	1-sample prophet	0.88	35.4	0.4	3.0	0.967	0.123
Classification (Gini)	ExtraTree ($m_{\text{try}} = 1$)	2.40	96.3	4.5	21.9	0.525	0.746
Classification (Gini)	ExtraTree ($m_{\text{try}} = p/3$)	2.09	89.2	1.9	8.3	0.779	0.721
Classification (Gini)	ExtraTree ($m_{\text{try}} = 2p/3$)	1.65	84.0	1.1	3.6	0.943	0.697
Classification (Gini)	ExtraTree ($m_{\text{try}} = p$)	1.44	84.6	0.4	2.6	0.967	0.590
Classification (Entropy)	Exhaustive	1.00	0.0	0.0	0.0	1.000	0.000
Classification (Entropy)	S (n/e)	0.66	-13.9	0.0	0.8	1.000	0.000
Classification (Entropy)	S^2 (n/e)	1.41	37.3	1.6	10.9	0.738	0.574
Classification (Entropy)	S_{all} (n/e)	1.83	59.5	1.6	10.6	0.746	0.705
Classification (Entropy)	S_{par} $(10, q = 0.5)$	0.89	28.2	0.6	2.6	0.959	0.139
Classification (Entropy)	Block-rank	1.93	70.8	2.3	14.3	0.631	0.713
Classification (Entropy)	1-sample prophet	0.94	35.4	0.4	2.9	0.984	0.148
Classification (Entropy)	ExtraTree ($m_{\text{try}} = 1$)	2.45	95.8	4.5	22.3	0.516	0.730
Classification (Entropy)	ExtraTree ($m_{\text{try}} = p/3$)	2.09	88.5	1.8	7.4	0.787	0.730
Classification (Entropy)	ExtraTree ($m_{\text{try}} = 2p/3$)	1.71	83.8	1.1	4.0	0.943	0.697
Classification (Entropy)	ExtraTree ($m_{\text{try}} = p$)	1.44	83.4	0.5	2.4	0.975	0.598

overhead can erase most of the nominal effort reduction.

The classification results remain more favorable than the regression results, but they also sharpen the distinction between quality-preserving and speed-preserving approximations. The optimized split-level secretary and prophet variants now achieve near-zero median loss: for both Gini and entropy, $S(n/e)$, prophet, and $S_{\text{par}}(10, q = 0.5)$ stay below 0.6% median loss. However, these methods no longer deliver strong wall-clock gains, with median speedups below one in all three cases. The practical speed-oriented compromises are instead ExtraTree($m_{\text{try}} = p/3$), ExtraTree($m_{\text{try}} = 2p/3$), $S_{\text{all}}(n/e)$, and block-rank, which trade roughly 1–2.5% median loss for 1.7× to 2.1× speedups. Supplementary Figure S2 still shows that S_{par} can be interesting on small classification datasets, but globally the optimized split-level rules primarily demonstrate how much of the original loss gap came from the selection implementation rather than from the stopping principle itself.

A plausible explanation for the strong task asymmetry observed in the benchmark is that classification impurities are more tolerant to approximate split search than regression impurity. For Gini and entropy, many nearby thresholds can induce very similar child class proportions and therefore nearly equivalent impurity reductions. In such cases, partial random exploration may still identify a split whose downstream predictive effect is close to that of the exhaustive optimum. By contrast, regression gains based on MSE appear to depend more sharply on the precise threshold location, because small changes in the partition can alter child means and variances continuously and hence affect the final predictions more directly. Supplementary Figure S13 examines this interpretation directly under exhaustive search by comparing the mass of thresholds within 5% and 10% of the nodewise optimum, the gap between the best gain and the median random-threshold gain, and the width of the near-optimal region on the winning covariate. The resulting comparison remains more nuanced than a single scalar notion of flatness: some diagnostics are larger for regression, whereas the best-versus-random gap is larger for classification. This suggests that the empirical advantage of classification is not explained by one universal notion of broader near-optimal threshold regions alone, and likely reflects a combination of impurity geometry, competition across covariates, downstream loss sensitivity, and the cost of exact replay.

3.3. Reliability and dataset structure

Figure 2 complements the median-based summaries by showing reliability under explicit operating constraints. The more aggressive schedules within the split-level secretary families can satisfy moderate loss tolerances on a non-negligible subset of runs, especially in classification, but they also exhibit heavier tails of poor outcomes. Size-stratified reliability curves are reported in Supplementary Figure S6, and the full grid of joint success probabilities over all datasets is reported in Supplementary Figure S7.

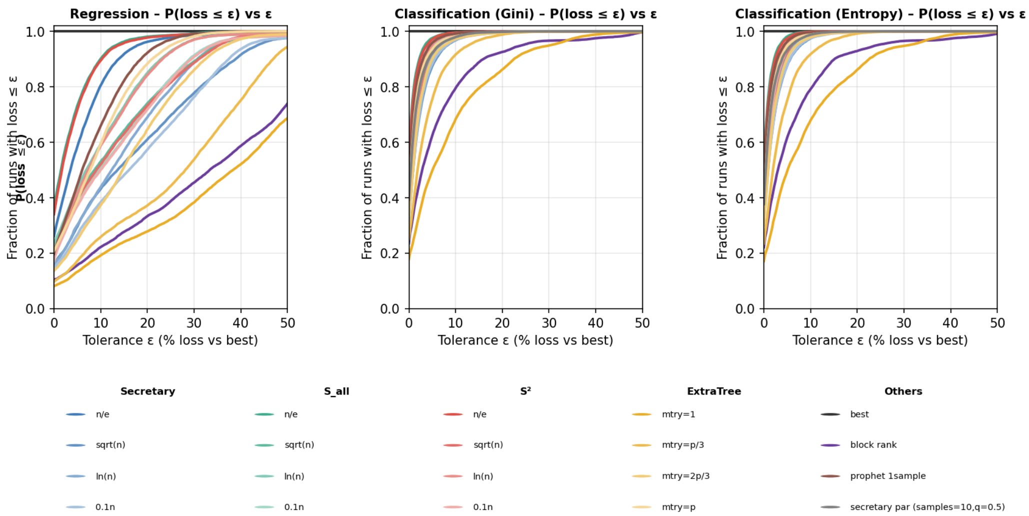

Figure 2: Reliability summaries for secretary-style split-search methods. Top row: empirical probability of achieving a predictive loss below a tolerance ε. Bottom panels: empirical probability of simultaneously achieving at least a prescribed time saving and a predictive loss below a prescribed tolerance. Separate panels are shown for regression, classification with Gini impurity, and classification with entropy impurity. Size-stratified and method-specific variants are reported in the supplementary material.

By contrast, S_{all} and block-rank are less aggressive but more reliable, and the optimized $S(n/e)$ and prophet variants sit at the opposite extreme: they preserve loss well but offer little speedup. This helps explain why the families rank differently depending on whether the target is strict reliability, strict runtime savings, or near-exhaustive predictive preservation. Supplementary Figures S3–S5 report task-specific ridgeline summaries of within-dataset variability, and Supplementary Figures S9–S11 provide complementary whisker summaries of the same repeated-run variability.

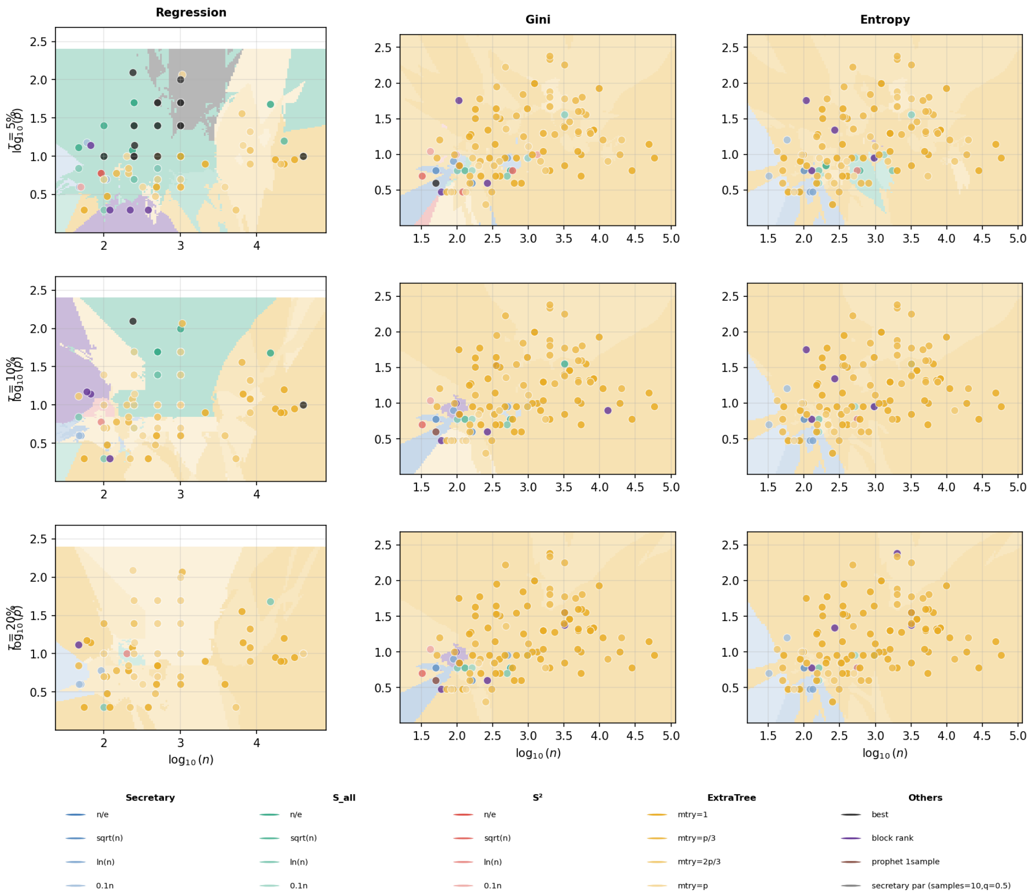

Figure 3: Regime maps as a function of dataset size and dimensionality. Each point is a benchmark dataset represented in the $(\log_{10} n, \log_{10} p)$ plane and colored by the split-search strategy that gives the best practical trade-off under a fixed speedup–loss criterion. Columns correspond to loss tolerances $\tau = 5\%, 10\%, 20\%$. Rows correspond to regression, classification with Gini impurity, and classification with entropy impurity.

Figure 3 shows that the usefulness of secretary-style split search depends on dataset structure. Classification tasks are clearly more favorable than regression tasks. Within classification, and under the chosen speedup–loss criterion, covariate-level and rank-adaptive methods dominate larger portions of the regime map than split-level secretary rules. In regression, exhaustive split search remains the safest choice on much of the benchmark, and secretary-style rules are only competitive in limited regions.

3.4. Sensitivity and negative result for the parametric strategy

We examined four exploration schedules based on node size: n/e, $0.1n$, $\log n$, and $\sqrt{n}$. The full results are reported in the supplementary material. Changing the schedule affects the quantitative trade-off more than the qualitative ranking. For S and S^2, more aggressive schedules produce slightly larger speedups at the price of slightly worse predictive degradation. For S_{all}, the n/e schedule consistently provides the best compromise, especially in classification, which justifies its use as the default representative in the main text. The corresponding size-stratified Pareto analyses, reliability summaries, and effort comparisons are reported in Supplementary Figures S1, S6, S7, S14, and S15.

The parametric secretary method S_{par} was evaluated in an initial screening study over a grid of fitting sample sizes and quantile levels. Most configurations are dominated in practice: the cost of fitting the working model outweighs any benefit from early stopping, especially in regression and on larger classification datasets. One low-overhead classification configuration, however, remains competitive on small datasets, which is why we retain $S_{\text{par}}(10, q = 0.5)$ as a representative line in the main classification figures and table. Even there, its global performance remains less robust than the strongest ExtraTrees, S_{all}, and block-rank baselines. This mostly negative result shows that probabilistic calibration is not useful by itself when the fitting overhead is too large relative to the gain-computation cost.

For orientation, the supplementary material is organized as follows: Supplementary Figure S1 gives the size-stratified Pareto comparison; Supplementary Figure S2 gives the S_{par} screening study; Supplementary Figures S3–S5 give the ridgeline summaries; Supplementary Figures S6–S7 and S12 give the reliability and joint-success plots; Supplementary Figure S8 gives the regime maps; Supplementary Figures S9–S11 give the within-dataset whisker summaries; Supplementary Figure S13 gives exhaustive gain-landscape diagnostics for the regression–classification asymmetry; and Supplementary Figures S14–S15 compare split-search effort across methods and against wall-clock savings.

4. Conclusion, limitations, and outlook

This paper asked whether exhaustive CART split search can be replaced, at least in part, by secretary-style sequential selection without incurring an unacceptable loss in predictive performance. The answer depends strongly on both the stopping rule and the prediction task, and it also depends on implementation details. After aligning exploration with ExtraTrees-style continuous threshold draws and optimizing the exact-selection phase, the simple secretary and prophet variants preserve predictive performance much more closely than in the earlier implementation, but they do so with little wall-clock gain. The methods that still offer the clearest practical speedups are the ExtraTrees baselines, the covariate-level strategy S_{all}, and the rank-adaptive block-rank rule.

The study reveals a clear task asymmetry. Regression trees are much less tolerant to incomplete split exploration than classification trees if one insists on real time savings. On the regression benchmarks considered here, the only low-loss secretary-style variants are effectively near-exhaustive in runtime, whereas the faster methods incur substantial loss. In classification, by contrast, the losses of several approximate methods remain modest enough to make the time–accuracy trade-off practically interesting. This suggests that the value of approximate split search depends both on the impurity criterion and on how efficiently a given stopping rule can convert reduced gain-evaluation effort into actual runtime savings.

A second important outcome is mostly negative but informative. The parametric secretary variant S_{par}, although statistically motivated by gain-distribution approximations, is not competitive globally once fitting overhead is taken into account. One low-overhead configuration remains interesting on small classification datasets, but overall the family is less robust than the strongest ExtraTrees, S_{all}, and block-rank baselines. This cautions against assuming that a principled probabilistic calibration will automatically lead to an effective split-search procedure.

The study is limited to single trees, benchmark datasets, and a deliberately small family of stopping rules, and is intended primarily as an empirical characterization rather than a complete theory. This restriction is deliberate: understanding the time–loss trade-off for a single tree is a necessary first step before one can interpret what these splitters might contribute inside larger ensembles.

A natural next step is to study the same split-search rules in bagging and boosting, but this remains genuine future work rather than a claim of the present paper. Because split-search cost is incurred at every node and across many trees, even moderate node-level savings may accumulate into substantial end-to-end gains, and ensembling may damp the predictive effect of occasional suboptimal splits. At the same time, those ensemble-level effects cannot be interpreted cleanly without first understanding the single-tree regime studied here. A second direction is methodological: the performance of the block-rank rule suggests that adaptive rank-based or sample-based stopping rules may be preferable to the classical secretary rule in this context. Overall, secretary-style split search is not a universal replacement for exhaustive CART search, but in favorable settings, especially in classification, it offers a simple and interpretable way to trade split-search time against predictive performance.

References

[1] L. Breiman, J. H. Friedman, R. A. Olshen, C. J. Stone, Classification and Regression Trees, Wadsworth, Belmont, CA, 1984.

[2] L. Breiman, Random forests, Machine Learning 45 (1) (2001) 5–32. doi:10.1023/A:1010933404324.

[3] P. Geurts, D. Ernst, L. Wehenkel, Extremely randomized trees, Machine Learning 63 (1) (2006) 3–42. doi:10.1007/s10994-006-6226-1.

[4] J. P. Gilbert, F. Mosteller, Recognizing the maximum of a sequence, Journal of the American Statistical Association 61 (313) (1966) 35–73. doi:10.1080/01621459.1966.10502008.

[5] T. S. Ferguson, Who solved the secretary problem?, Statistical Science 4 (3) (1989) 282–296. doi:10.1214/ss/1177012493.

[6] P. Nuti, J. Vondrák, Secretary problems: The power of a single sample, in: Proceedings of the 2023 ACM-SIAM Symposium on Discrete Algorithms, SIAM, 2023, pp. 2015–2029. doi:10.1137/1.9781611977554.ch77.

[7] J. D. Romano, T. T. Le, W. L. Cava, J. T. Gregg, D. J. Goldberg, N. L. Ray, P. Chakraborty, D. Himmelstein, W. Fu, J. H. Moore, Pmlb v1.0: an open-source dataset collection for benchmarking machine learning methods, Bioinformatics 38 (3) (2022) 878–880. doi:10.1093/bioinformatics/btab665.

[8] A. Rubinstein, J. Z. Wang, S. M. Weinberg, Optimal single-choice prophet inequalities from samples, in: 11th Innovations in Theoretical Computer Science Conference (ITCS 2020), Vol. 151 of LIPIcs, Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2020, pp. 60:1–60:10. doi:10.4230/LIPIcs.ITCS.2020.60.

[9] E. Samuel-Cahn, Comparison of threshold stop rules and maximum for independent nonnegative random variables, The Annals of Probability 12 (4) (1984) 1213–1216. doi:10.1214/aop/1176993150.

[10] B. Laurent, P. Massart, Adaptive estimation of a quadratic functional by model selection, The Annals of Statistics 28 (5) (2000) 1302–1338. doi:10.1214/aos/1015957395.